

Definitive OMI Address Schema for JSON Canvas Color/Emoji/Incidence Graphs

1. Core Claim

OMI should encode JSON Canvas world state as a graph:

$$G = (V, E, I)$$

Where:

V = vertices / nodes / color-body objects
E = edges / relations / reader routes
I = incidence / side attachments / source-plane crossings

The color-address function is:

$$G(AA) = RRGGBB$$

Meaning:

AA selects or scopes a color/address/incidence attachment.
RRGGBB is the lower structural 24-bit color body.

The canonical 32-bit seed is:

$$RRGGBBAA$$

Where:

RRGGBB = lower structural color body
AA = address attachment / incidence selector / emoji-CIDR lens byte

The orbit recovery law is:

```
orbit, offset = divmod(0xRRGGBBAA, 36)
```

This is the simplest ubiquitous form because:

```
RRGGBB = 24 bits  
AA      = 8 bits  
total   = 32 bits
```

So the OMI canvas address uses a standard 32-bit carrier while preserving the lower/upper boundary.

2. Lower vs Upper Color Law

OMI separates two color authorities.

Lower structural color

The lower stack uses full hex colors:

```
#RRGGBB
```

This belongs to:

```
≤ 8!  
Omicron frame  
Delta Law  
BQF body  
Q_frame  
Q_xy  
atomic replay  
JSON Canvas geometry
```

Full six-digit hex color is exact structural state.

It is not merely a theme.

Upper reader color

The upper stack uses preset color keys:

```
"1" red
"2" orange
"3" yellow
"4" green
"5" cyan
"6" purple
```

These are cosmetic reader keys.

They belong to:

```
9!-12!
unicode-bidi
reader lens
observer shell
template phase
display orientation
```

Therefore:

```
Lower hex encodes.
Upper presets display.
```

3. The RRGGBBAA Seed

The canonical seed is:

```
C32 = 0xRRGGBBAA
```

Where:

```
RR = red channel  0x00..0xFF
GG = green channel 0x00..0xFF
BB = blue channel 0x00..0xFF
AA = address attachment byte
```

The lower body is:

```
RGB24 = 0xRRGGBB
```

The upper/lens/incidence byte is:

```
AA = 0x00..0xFF
```

OMI does not treat `AA` as ordinary alpha by default.

In this schema, `AA` means:

```
A = address  
A = attachment  
A = aperture  
A = annotation
```

If a renderer wants visual alpha, it may derive alpha as a reader lens, but the canonical OMI meaning of `AA` is address/incidence attachment.

4. Base36 Orbit Recovery

The Delta Law gives:

```
period = 8  
prime = 73  
block B = [0,1,3,6,9,8,6,3]  
W = 36
```

Therefore the canonical orbit recovery is:

```
orbit = floor(C32 / 36)  
offset = C32 mod 36
```

Or:

```
orbit, offset = divmod(C32, 36)
```

Where:

offset $\in 0..35$

The offset maps to Base36:

0..9 $\rightarrow$ 0..9
10..35 $\rightarrow$ A..Z

This preserves:

Delta Law $\rightarrow$ period 8
1/73 block $\rightarrow$ W = 36
Base36 $\rightarrow$ visible orbit alphabet

The color-address seed does not create the orbit.

It reads the orbit from a valid encoded state.

5. The 37 Mode-Switch Sentinel

Base36 uses exactly:

0..35

Therefore OMI reserves:

37

as the first safe mode-switch sentinel outside the Base36 orbit alphabet.

This prevents collision with:

offset = C32 mod 36

Because `divmod(..., 36)` can never return 37 as an offset.

So:

```
0..35 = Base36 orbit offsets
36    = width boundary / W marker
37    = mode-switch sentinel
```

The rule is:

```
37 is not an orbit offset.
37 is an Omicron mode switch.
```

This allows OMI to support irrational, template, or reader-mode transitions without corrupting Base36 orbit recovery.

6. OMI Address Form

The canonical JSON Canvas OMI address should be:

```
omi-<frame>/128/@rgb:<RRGGBB>/@aa:<AA>/@36:<offset>
```

Example:

```
omi-0400-03bf-0003-2b04-2f04-0002-039f-04ff/128/@rgb:FF0000/@aa:25/@36:X
```

Where:

```
omi-0400-03bf-0003-2b04-2f04-0002-039f-04ff
  = core self-sufficient OMI address

/128
  = exact claim boundary

/@rgb:FF0000
  = lower structural color body

/@aa:25
  = address attachment byte

/@36:X
  = Base36 orbit offset label
```

If `AA = 0x25`, decimal `37`, then the parser treats it as:

```
mode-switch sentinel
```

not as a Base36 offset.

7. JSON Canvas Mapping

A JSON Canvas node maps to an OMI vertex.

```
{
  "id": "omi-.../128/@rgb:FF0000/@aa:00/@36:0",
  "type": "text",
  "x": 0,
  "y": 0,
  "width": 240,
  "height": 120,
  "color": "#FF0000",
  "text": "Logos"
}
```

OMI interpretation:

```
id
  principal OMI pointer

type
  node kind / world object type

x,y
  Q_xy projection plane

width
  local240 / bridge width

height
  hidden 5! root depth

color: "#RRGGBB"
  lower structural color body
```

A JSON Canvas edge maps to an OMI incidence relation.

```
{
  "id": "omi-edge-...",
  "fromNode": "omi-node-a",
  "fromSide": "right",
  "toNode": "omi-node-b",
  "toSide": "left",
  "toEnd": "arrow",
  "color": "1",
  "label": "reclaims"
}
```

OMI interpretation:

```
fromNode / toNode
  source and target OMI vertices

fromSide / toSide
  tangent-crossing port

toEnd
  sequencing direction

color: "1"
  upper reader color key

label
  predicate / 0-expression relation
```

8. $G = (V, E, I)$ Schema

The graph is:

```
G = (V, E, I)
```

8.1 Vertices

Each vertex is:

```
v ∈ V
```

with:


```

v.id      = OMI pointer
v.color   = #RRGGBB
v.seed32  = 0xRRGGBBAA
v.orbit   = floor(seed32 / 36)
v.offset  = seed32 mod 36
v.label36 = Base36[offset]

```

8.2 Edges

Each edge is:

```
e ∈ E
```

with:

```

e.fromNode
e.toNode
e.fromSide
e.toSide
e.label
e.color

```

Edges are not merely visual lines.

They are OMI relation routes.

8.3 Incidence

Each incidence is:

```
i ∈ I
```

with:

```
i = (vertex, edge, side, attachment)
```

The **AA** byte can identify the incidence attachment:

```
AA = incidence selector
```

So:

```
G(AA) = RRGGBB
```

means:

```
given an incidence/address attachment byte,  
recover the lower structural color body.
```

9. Miquel Configuration Model

The Miquel configuration gives the right geometric analogy:

```
8 points  
6 circles  
each point lies on 3 circles  
each circle passes through 4 points
```

OMI maps this to the RGB cube.

The 8 cube vertices are the binary RGB corners:

```
#000000  
#FF0000  
#00FF00  
#0000FF  
#FFFF00  
#FF00FF  
#00FFFF  
#FFFFFF
```

The 6 circles correspond to the 6 cube faces / channel constraints:

```
R = 0  
R = 255  
G = 0  
G = 255  
B = 0  
B = 255
```

Each color vertex lies on 3 channel faces.

Each channel face contains 4 color vertices.

So the color cube naturally realizes:

```
8 vertices × 3 incidences = 24
6 faces × 4 incidences = 24
```

This mirrors the Miquel-style incidence structure:

```
(8_3, 6_4)
```

In OMI terms:

```
8 color vertices
6 channel circles
24 incidence attachments
```

The `AA` byte can route which incidence attachment is active.

10. Relation to UTF-EBCDIC as Process Model

The UTF-EBCDIC analogy is useful as a process model:

```
a byte-oriented transport layer
maps character identity through an encoded carrier
without losing the distinction between character, byte, and interpretation.
```

OMI uses the same kind of discipline:

```
RRGGBB
  lower structural body

AA
  attachment / carrier / address lens

RRGGBBAA
  compact 32-bit transport seed
```

```
divmod(seed,36)
Base36 orbit recovery
```

The carrier is compact, but the interpretation remains layered.

The seed does not erase the object model.

It gives the object a small universal addressable projection.

11. Lower 8! and Upper 9! Boundary

The lower side:

```
RRGGBB
#RRGGBB
Omicron
Delta Law
BQF
Q_frame
Q_xy
terminating fractions
```

The upper side:

```
AA
preset color keys
emoji CIDR lenses
unicode-bidi
reader direction
template phase
repeating fractions
```

But the boundary is strict:

```
AA may select a reader or incidence lens.
AA may not validate the lower RRGGBB body.
AA may not mutate the Omicron frame.
AA may not create the creation step.
```

So:

RRGGBB = body
AA = attachment

12. Emoji CIDR Attachment

The phrase:

AA: cidr(emoji) of cidr of lower 8! RRGGBB

becomes:

AA is a compact attachment byte that points from lower RGB body state to an emoji/CIDR reader lens.

So an OMI node may carry:

/@rgb:FF0000
/@aa:25
/@emoji:🔥

The emoji is not authority.

The emoji is the visible symbolic carrier.

The lower body remains:

#FF0000

The upper reader may display:



But the canonical seed remains:

0xFF000025

13. Color Count

The lower RGB body has:

$$256 \times 256 \times 256 = 16,777,216$$

possible structural colors.

This is:

$$2^8 \times 2^8 \times 2^8 = 2^{24}$$

Adding `AA` yields:

$$2^{24} \times 2^8 = 2^{32}$$

possible color-address seeds.

Therefore:

RRGGBB gives exact lower color state.
RRGGBBAA gives exact lower color state plus one byte of incidence/reader attachment.

14. Canonical Parser Contract

A parser should process a JSON Canvas OMI node in this order:

1. Read `node.id` as OMI pointer.
2. Validate core Omicron frame.
3. Read color.
4. If color is `#RRGGBB`:
 classify as lower structural color.
5. If color is `"1".."6"`:
 classify as upper reader color.
6. Read `AA` attachment if present.
7. Compute `seed32 = 0xRRGGBBAA`.
8. Compute orbit, `offset = divmod(seed32, 36)`.
9. If `AA = 37`:

```
    trigger Omicron mode switch.  
10. Emit vertex/incidence record.
```

The parser must never allow:

```
preset color validates object  
emoji validates object  
AA validates object  
unicode-bidi validates object
```

Only the OMI frame validates the object.

15. Canonical JavaScript Shape

```
{  
  omi: "omi-0400-03bf-0003-2b04-2f04-0002-039f-04ff",  
  claim: 128,  
  rgb: "FF0000",  
  aa: "25",  
  seed32: 0xFF000025,  
  orbit: Math.floor(0xFF000025 / 36),  
  offset: 0xFF000025 % 36,  
  base36: "X",  
  modeSwitch: true,  
  colorAuthority: "lower-structural",  
  emojiAuthority: "projection-only"  
}
```

16. Rules

Add these rules:

```
# [Rule 0xC3]: JSON Canvas RGB-AA Address Seed  
omi-0000-0000-0000-0000-0000-0000-00c3-0001/128 MUST encode-json-canvas-color-  
addresses-as-rrggbbaa-seeds  
  
# [Rule 0xC4]: Base36 Color Orbit Recovery  
omi-0000-0000-0000-0000-0000-0000-00c4-0001/128 MUST recover-json-canvas-color-  
orbits-through-divmod-rrggbbaa-by-thirty-six
```

```
# [Rule 0xC5]: AA Mode-Switch Sentinel
omi-0000-0000-0000-0000-0000-0000-00c5-0001/128 MUST reserve-thirty-seven-as-aa-
mode-switch-outside-base36-offsets

# [Rule 0xC6]: Miquel RGB Incidence Schema
omi-0000-0000-0000-0000-0000-0000-00c6-0001/128 MUST map-json-canvas-rgb-color-
cube-through-eight-vertex-six-face-incidence
```

17. Facts

After implementation:

```
# [Fact 0xC3-1001]: RGB-AA Seed Implemented
omi-0000-0000-0000-0000-0000-0000-00c3-1001/128 FACT json-canvas-rrgbbaa-seed-
implemented

# [Fact 0xC4-1001]: Color Orbit Recovery Implemented
omi-0000-0000-0000-0000-0000-0000-00c4-1001/128 FACT json-canvas-color-orbit-
recovery-divmod36-implemented

# [Fact 0xC5-1001]: AA Mode Switch Implemented
omi-0000-0000-0000-0000-0000-0000-00c5-1001/128 FACT aa-thirty-seven-mode-
switch-sentinel-implemented

# [Fact 0xC6-1001]: Miquel RGB Incidence Implemented
omi-0000-0000-0000-0000-0000-0000-00c6-1001/128 FACT miquel-rgb-eight-vertex-
six-face-incidence-implemented
```

18. Implementation Modules

Create:

```
src/canvas/json-canvas-address-schema.js
src/canvas/rrgbbaa-orbit.js
src/canvas/miquel-rgb-incidence.js
```

Suggested exports:

```
parseRgbHex(color)
parseCanvasPresetColor(color)
```



```
composeRRGGBBAA(rgb, aa)
recoverRRGGBBAAOrbit(seed32)
isAAModeSwitch(aa)
classifyCanvasColorAuthority(color)
buildJsonCanvasOmiVertex(node)
buildJsonCanvasOmiEdge(edge)
buildMiquelRgbIncidence()
```

19. Required Tests

Add:

```
test/json-canvas-address-schema.test.js
test/rrggbbaa-orbit.test.js
test/miquel-rgb-incidence.test.js
```

Test requirements:

```
#FF0000 parses as lower structural RGB.
"1" parses as upper preset reader color.
RRGGBB composes with AA into 32-bit seed.
divmod(RRGGBBAA,36) returns orbit and offset.
Offset is always 0..35.
AA decimal 37 triggers modeSwitch true.
AA mode switch does not collide with offset.
8 RGB cube vertices are generated.
6 RGB channel faces are generated.
24 Miquel incidences are generated.
Each RGB vertex touches 3 channel faces.
Each channel face contains 4 RGB vertices.
JSON Canvas geometry remains structural.
Preset colors remain reader/cosmetic.
Emoji carrier remains projection-only.
```

20. Final Doctrine

OMI encodes JSON Canvas state through $G = (V, E, I)$, where vertices carry lower structural #RRGGBB color bodies, edges carry relation routes, and incidences carry AA attachment bytes. The canonical seed RRGGBBAA is recovered through

`divmod(seed,36)`, preserving the Delta-derived Base36 orbit while reserving 37 as an Omicron mode-switch sentinel outside the Base36 offset space.

Short form:

`G = (V,E,I).`
`G(AA) = RRGGBB.`
RRGGBBAA seeds the orbit.
`divmod(seed,36)` recovers position.
37 switches mode.
Hex encodes lower body.
Preset keys display upper reader.